

UNIVERSIDADE FEDERAL DE ITAJUBÁ - UNIFEI
CAMPUS ITABIRA
ENGENHARIA DE COMPUTAÇÃO

ECO12209 - LINGUAGENS DE PROGRAMAÇÃO

Projeto Prático: Dashboard de Indicadores Financeiros Públicos

Equipe Discente:

Eduardo Antônio Lameira dos Anjos - 2024014417
d2024014417@unifei.edu.br

Felipe Scannavino Sakashita - 2024013278
d2024013278@unifei.edu.br

Gabriela Portugal Rocha Lopes - 2024009471
d2024009471@unifei.edu.br

Otávio Lima de Andrade - 2024003487
d2024003487@unifei.edu.br

Docente: Prof. Walter Aoiama Nagai

Itabira, MG
Junho de 2026

1 Introdução e Definição do Problema

O projeto consiste na concepção e desenvolvimento de um **Dashboard de Indicadores Financeiros Públicos**, alinhado ao Tema 5 do Projeto Prático da disciplina ECO12209. O objetivo central é fornecer uma ferramenta consolidada para o acompanhamento ágil de índices econômicos críticos (como IPCA, Selic e Câmbio), auxiliando a tomada de decisão técnica na gestão pública e promovendo transparência por meio de dados abertos.

Em conformidade com os princípios obrigatórios da disciplina, o sistema adota uma arquitetura poliglota que explora três paradigmas distintos e complementares de programação: **Rust**, **Go** e **Dart**, garantindo que o processamento seja feito em tempo real com fontes de dados verídicas e com alta performance.

2 Arquitetura e Integração das Linguagens

As responsabilidades técnicas do ecossistema foram divididas para extrair o máximo de desempenho de cada tecnologia:

- **Rust (Módulo de Coleta e Ingestão):** Designado para o trabalho pesado de consumo de APIs externas. O Rust atua com rotinas assíncronas (`reqwest`, `tokio`) para buscar, em tempo real, as variações cambiais e indexadores governamentais, garantindo um parsing veloz, tolerância a falhas e segurança de memória.
- **Go (Servidor e API RESTful):** Atua como a camada central de orquestração. O servidor Go é responsável por expor rotas protegidas para a aplicação cliente, processar requisições em alta concorrência utilizando *goroutines* e fornecer dados formatados em JSON para o frontend.
- **Dart / Flutter (Interface Gráfica Frontend):** Focado na experiência do usuário. Apresenta um painel visual responsivo e dinâmico, exibindo gráficos de variação temporal e painéis com métricas-chave atualizadas via requisições assíncronas à API em Go.

3 APIs Públicas Adotadas

O protótipo captura dados abertos em tempo real através dos seguintes *endpoints* e provedores:

Indicador	Provedor / API	Justificativa
Dólar (USD-BRL)	AwesomeAPI	Métrica global crítica para transações e importações.
Taxa Selic	SGS / Banco Central	Principal instrumento de controle inflacionário.
IPCA Mensal	SGS / Banco Central	Indexador essencial para contratos públicos e de mercado.

4 Containerização com Docker

Recentemente, o projeto recebeu uma importante atualização estrutural visando facilitar o processo de *deploy*, orquestração e testes: a implementação de um **ambiente Docker**.

Através da containerização, os serviços em Rust (coleta) e Go (orquestração da API) foram encapsulados em imagens enxutas e independentes. O uso do Docker e do `docker-compose` solucionou problemas de conflito de ambiente e dependências, permitindo que a aplicação poliglota seja levantada de maneira unificada e padronizada com um único comando. Além disso, garante que o ambiente de desenvolvimento seja o mesmo do ambiente de produção.

5 Resultados Alcançados e Desafios Técnicos

O desenvolvimento deste sistema proporcionou desafios significativos na integração entre linguagens. O principal desafio técnico superado foi a implementação da comunicação eficiente entre o módulo de ingestão em **Rust** e o orquestrador em **Go**, garantindo a atomicidade e consistência dos dados financeiros coletados.

A escolha do Flutter (Dart) como camada de apresentação provou-se acertada, permitindo a criação de um layout moderno e responsivo que consome a API de forma assíncrona, mantendo a fluidez da interface. Atualmente, o sistema encontra-se perfeitamente funcional em ambiente local (via containers), demonstrando a viabilidade, escalabilidade e facilidade de manutenção de uma arquitetura poliglota para aplicações de alta responsabilidade.

6 Organização do Repositório (Git)

O repositório mantém uma estrutura fragmentada, focada em isolamento de domínios e microsserviços. Os arquivos estão organizados da seguinte forma:

- `/Rust/`: Contém os *scripts* de coleta de dados, códigos-fonte `.rs` e manifestos Cargo.
- `/GO/`: Reserva a lógica de roteamento web, regras de negócios e serviços REST em Go.
- `/Dart/`: Abriga os componentes visuais, bibliotecas (lib) e controle de estado do aplicativo (Flutter).
- `docker-compose.yml` e `Dockerfiles`: Na raiz do repositório (ou em seus respectivos diretórios), responsáveis pela subida orquestrada dos ambientes Rust e Go.
- Arquivos de configuração globais (`.env`, `.gitignore`, `README.md`).

7 Ética no Uso de Ferramentas de IA

O grupo atesta que qualquer adoção de modelos de Inteligência Artificial Generativa baseou-se nos princípios obrigatórios de transparência e integridade acadêmica. A lógica analítica, a interconexão de redes (HTTP), os componentes visuais, os scripts *Docker* e

o pensamento arquitetural foram integralmente concebidos e implementados de maneira original pelos discentes.